

Table of Contents

What is the homogeneous transform that converts any point from room frame to image frame ?

Given Information:

Part 1

Exact Homogeneous Transform from Given Data

Main Solution:

Verification 1:

Verification 2:

Part 2

Registration Method

Main Solution:

Verification 1:

Display of Frames

```
clear all;clc;
```

What is the homogeneous transform that converts any point from room frame $[O_0]$ to image frame $[O_I]$?

Given Information:

$$\begin{matrix} \rightarrow \\ (O_0 O_t)^0 \end{matrix} = \begin{pmatrix} 0.2 \\ 4.1 \\ -0.1 \end{pmatrix}, \quad \text{Euler}_{\text{XYX}} = 25^\circ, 15^\circ, 3^\circ$$

```
Ed = [25 15 3];           % Euler angles (XYX convention) [deg]
Er = deg2rad(Ed);         % Euler angles (XYX convention) [rad]
P_0t = [.2; 4.1; -.1];    % Translation  $O_0 O_t$  (room frame)
```

Part 1

Exact Homogeneous Transform from Given Data

Main Solution:

$$H_0^I = (H_I^0)^{-1}$$

The homogeneous transform is assembled as: $H_i^j = \begin{bmatrix} R_i^j & O_j O_i \\ \emptyset & 1 \end{bmatrix}$.

For a homogeneous transform with rotation R_i^j and translation $O_j O_i$, the inverse has a closed-form:

$$(H_i^j)^{-1} = H_j^i = \begin{bmatrix} (R_i^j)^T & -(R_i^j)^T \cdot O_j O_i \\ \emptyset & 1 \end{bmatrix}.$$

$$\text{So, } H_I^0 = \begin{bmatrix} R_I^0 & (O_0 O_I)^0 \\ \emptyset & 1 \end{bmatrix}.$$

Where:

- $R_I^0 = R_I^0 = R_{x,25^\circ} R_{x,15^\circ} R_{x,3^\circ}$ defined using Euler XYZ sequence, first rotate 25° around X, then 15° around new Y, then 3° around new X.

$$R_{x,\theta} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos(\theta) & -\sin(\theta) \\ 0 & \sin(\theta) & \cos(\theta) \end{pmatrix} \quad R_{y,\theta} = \begin{pmatrix} \cos(\theta) & 0 & \sin(\theta) \\ 0 & 1 & 0 \\ -\sin(\theta) & 0 & \cos(\theta) \end{pmatrix}$$

- $(O_0 O_I)^0 = (O_0 O_I)^0 + R_I^0 (O_I O_I)^t$

```
% Rotation (tool → room)
```

```
R_t0 = Rx(Er(1))*Ry(Er(2))*Rx(Er(3))
```

```
R_t0 = 3x3
    0.9659    0.0135    0.2585
    0.1094    0.8837   -0.4551
   -0.2346    0.4679    0.8521
```

```
% Rotation (room → tool)
```

```
R_0t = R_t0.'
```

```
R_0t = 3x3
    0.9659    0.1094   -0.2346
    0.0135    0.8837    0.4679
    0.2585   -0.4551    0.8521
```

```
% Verification
```

```
det(R_t0)
```

```
ans = 1
```

```
R_t0*R_0t
```

```
ans = 3x3
    1.0000    0.0000         0
    0.0000    1.0000   -0.0000
         0   -0.0000    1.0000
```

```
% Rotation (image → room)
```

```
R_I0 = R_t0;
```

```
% Rotation (room → image)
```

```
R_0I = R_0t;
```

```
% Position  $O_i$ 
```

```
P_tI = [-.6;0;0]; % Translation  $O_t O_i$  (image frame)
```

```
P_0I = P_0t + R_t0*P_tI      % Translation 0_00_i (room frame)
```

```
P_0I = 3x1
-0.3796
 4.0344
 0.0407
```

```
% Calling ht function for Mapping between Room and image Frames
```

```
[H_I0,H_0I] = ht(R_I0,P_0I);
H_I0 = double(H_I0)
```

```
H_I0 = 4x4
 0.9659    0.0135    0.2585   -0.3796
 0.1094    0.8837   -0.4551    4.0344
-0.2346    0.4679    0.8521    0.0407
         0         0         0    1.0000
```

```
H_0I = double(H_0I)
```

```
H_0I = 4x4
 0.9659    0.1094   -0.2346   -0.0651
 0.0135    0.8837    0.4679   -3.5791
 0.2585   -0.4551    0.8521    1.8994
         0         0         0    1.0000
```

Verification 1:

- Using Basic Homogeneous Transforms

$$H_0^I = Trans_{x,0.6} Rot_{x,-3^\circ} Rot_{y,-15^\circ} Rot_{x,-25^\circ} Trans_{x,-0.2} Trans_{y,-4.1} Trans_{z,0.1}$$

$$Trans_x = \begin{pmatrix} 1 & 0 & 0 & x \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, Trans_y = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & y \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, Trans_z = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$Rot_x = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(\phi) & -\sin(\phi) & 0 \\ 0 & \sin(\phi) & \cos(\phi) & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, Rot_y = \begin{pmatrix} \cos(\theta) & 0 & \sin(\theta) & 0 \\ 0 & 1 & 0 & 0 \\ -\sin(\theta) & 0 & \cos(\theta) & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, Rot_z = \begin{pmatrix} \cos(\psi) & -\sin(\psi) & 0 & 0 \\ \sin(\psi) & \cos(\psi) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

```
H_0I_v1 = double(Transx(.6)*...
  Rotx(-Er(3))*Roty(-Er(2))*Rotx(-Er(1))*...
  Transx(-.2)*Transy(-4.1)*Transz(.1))
```

```
H_0I_v1 = 4x4
    0.9659    0.1094   -0.2346   -0.0651
    0.0135    0.8837    0.4679   -3.5791
    0.2585   -0.4551    0.8521    1.8994
         0         0         0         1.0000
```

Verification 2:

- Using auxiliary frame $[O_I]$

$$H_0^I = H_t^I H_0^t = (H_t^t)^{-1} (H_t^0)^{-1}$$

$$H_t^t = \begin{bmatrix} I_{3 \times 3} & O_t O_I \\ \emptyset & 1 \end{bmatrix}$$

% Calling ht function for Mapping between Room and Tool Frames

```
[H_t0,H_0t] = ht(R_t0,P_0t);
H_t0 = double(H_t0)
```

```
H_t0 = 4x4
    0.9659    0.0135    0.2585    0.2000
    0.1094    0.8837   -0.4551    4.1000
   -0.2346    0.4679    0.8521   -0.1000
         0         0         0         1.0000
```

```
H_0t = double(H_0t)
```

```
H_0t = 4x4
    0.9659    0.1094   -0.2346   -0.6651
    0.0135    0.8837    0.4679   -3.5791
    0.2585   -0.4551    0.8521    1.8994
         0         0         0         1.0000
```

% Calling ht function for Mapping between Tool and Image Frames

```
[H_It,H_tI] = ht(eye(3),P_tI);
H_It = double(H_It)
```

```
H_It = 4x4
    1.0000         0         0   -0.6000
         0    1.0000         0         0
         0         0    1.0000         0
         0         0         0         1.0000
```

```
H_tI = double(H_tI)
```

```
H_tI = 4x4
    1.0000         0         0    0.6000
         0    1.0000         0         0
         0         0    1.0000         0
         0         0         0         1.0000
```

$$H_{oI_v2} = H_{tI} * H_{\theta t}$$

$$H_{oI_v2} = 4 \times 4$$

0.9659	0.1094	-0.2346	-0.0651
0.0135	0.8837	0.4679	-3.5791
0.2585	-0.4551	0.8521	1.8994
0	0	0	1.0000

The homogeneous transformation H_0^I represents the room origin expressed in image coordinates, while H_0^t represents it in the tool frame. Since the tool frame is located 0.6 m ahead of the image frame along x-axis with same orientation, every point measured from room would be appear 0.6 m farther from the tool frame than from the image frame. So, the results are consistent, both transformations share the same rotation block, with the position vectors differing only by $+0.6\text{ m}$ along x-axis.

Part 2

Registration Method

In the graphics software, one corner of the box is at the origin of frame $[O_I]$. The box extends 0.6 m along X_I , 0.3 m along Y_I , and 0.2 m along Z_I . The 8 corners are all combinations of $\{0, 0.6\} \times \{0, 0.3\} \times \{0, 0.2\}$:

```
% Corner Coordinates in Image Frame (known from graphics model)
P_I = [0 0 0 0 .6 .6 .6 .6;...
       0 0 .3 .3 0 0 .3 .3;...
       0 .2 0 .2 0 .2 0 .2;
       1 1 1 1 1 1 1 1];
disp(table(P_I(:,1),P_I(:,2),P_I(:,3),P_I(:,4),P_I(:,5),P_I(:,6),P_I(:,7),P_I(:,8), ...
'VariableNames', {'P_1','P_2','P_3','P_4','P_5','P_6','P_7','P_8'}, ...
'RowNames', {'X_I','Y_I','Z_I','-'}))
```

	P_1	P_2	P_3	P_4	P_5	P_6	P_7	P_8
X_I	0	0	0	0	0.6	0.6	0.6	0.6
Y_I	0	0	0.3	0.3	0	0	0.3	0.3
Z_I	0	0.2	0	0.2	0	0.2	0	0.2
-	1	1	1	1	1	1	1	1

P_1 is origin of the image frame (placed at the graphic software's coordinate origin).

P_5 is origin of the tool frame (placed at the tool coordinate origin).

Navigation system measured the position of same 8 corner coordinates in $[O_0]$. (here they are calculated using H_I^0)

```
% Corner Coordinates in Room Frame (measured by navigation)
P_0 = H_I0*P_I;
disp(table(P_0(:,1),P_0(:,2),P_0(:,3),P_0(:,4),P_0(:,5),P_0(:,6),P_0(:,7),P_0(:,8), ...
'VariableNames', {'P_1','P_2','P_3','P_4','P_5','P_6','P_7','P_8'}, ...
'RowNames', {'X_I','Y_I','Z_I','-'}))
```

P_8	P_1	P_2	P_3	P_4	P_5	P_6	P_7
X_I	-0.37956	-0.32786	-0.37549	-0.3238	0.2	0.25169	0.20406
Y_I	4.0344	3.9434	4.2995	4.2085	4.1	4.009	4.3651
Z_I	0.040742	0.21116	0.1811	0.35152	-0.1	0.070422	0.040357
-	1	1	1	1	1	1	1

Main Solution:

H_0^I can be found by forming $\text{Mat}_P^I = H_0^I \text{Mat}_P^0$. Here there are 12 unknowns, 3 from position vector and 9 from rotation matrix. To solve for the unknowns 4 points needed to derive directly H_0^I .

$$H_0^I = \text{Mat}_P^I (\text{Mat}_P^0)^{-1}$$

Columns should be chosen as each position matrices be invertible. Physically means 4 non-coplanar points. One valid combination is P_1, P_5, P_6, P_8 .

```
Mat_I = [P_I(:,1) P_I(:,5) P_I(:,6) P_I(:,8)]
```

```
Mat_I = 4x4
    0    0.6000    0.6000    0.6000
    0         0         0    0.3000
    0         0    0.2000    0.2000
  1.0000    1.0000    1.0000    1.0000
```

```
Mat_0 = [P_0(:,1) P_0(:,5) P_0(:,6) P_0(:,8)]
```

```
Mat_0 = 4x4
 -0.3796    0.2000    0.2517    0.2558
  4.0344    4.1000    4.0090    4.2741
  0.0407   -0.1000    0.0704    0.2108
  1.0000    1.0000    1.0000    1.0000
```

```
h0I = Mat_I*inv(Mat_0)
```

```
h0I = 4x4
```

$$\begin{array}{cccc}
0.9659 & 0.1094 & -0.2346 & -0.0651 \\
0.0135 & 0.8837 & 0.4679 & -3.5791 \\
0.2585 & -0.4551 & 0.8521 & 1.8994 \\
0.0000 & -0.0000 & 0.0000 & 1.0000
\end{array}$$

Verification 1:

Alternatively, the 9 unknowns of rotation can be reduced to 6 by applying the properties of rotation matrix (the orthogonality and unit-length constraints). So, 3 non-colinear points can be used.

$$P_i^I = H_0^I P_i^0, \quad i = 1, 5, 3$$

$$\text{where } H_0^I = \begin{bmatrix} R_0^I & (O_I O_0)^I \\ \emptyset & 1 \end{bmatrix}.$$

$$E_1 : P_1^I = R_0^I P_1^0 + (O_I O_0)^I$$

$$E_2 : P_5^I = R_0^I P_5^0 + (O_I O_0)^I$$

$$E_3 : P_3^I = R_0^I P_3^0 + (O_I O_0)^I$$

$$\text{eliminating } (O_I O_0)^I \text{ as } P_1^I = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}:$$

$$E_2 - E_1 : P_5^I = R_0^I (P_5^0 - P_1^0) \quad \text{along } X_I \text{ axis} \rightarrow R_0^I P_5^I = |P_5^I| \hat{i}_1^0 = (P_5^0 - P_1^0)$$

$$E_3 - E_1 : P_3^I = R_0^I (P_3^0 - P_1^0) \quad \text{along } Y_I \text{ axis} \rightarrow R_0^I P_3^I = |P_3^I| \hat{j}_1^0 = (P_3^0 - P_1^0)$$

$$\begin{aligned}
\hat{i}_1^0 &= \frac{(P_5^0 - P_1^0)}{|P_5^I|} \\
\hat{j}_1^0 &= \frac{(P_3^0 - P_1^0)}{|P_3^I|} \\
\hat{k}_1^0 &= \hat{i}_1^0 \times \hat{j}_1^0
\end{aligned}
\rightarrow R_0^I = \begin{pmatrix} \hat{i}_1^0 \\ \hat{j}_1^0 \\ \hat{k}_1^0 \end{pmatrix}$$

Solving $(O_I O_0)^I$:

$$(O_I O_0)^I = P_1^I - R_0^I P_1^0$$

% Calculating unit vectors of [O_I] expressed in [O_0]

```

i_I0 = [P_0(1:3,5)-P_0(1:3,1)]/norm(P_I(1:3,5));
j_I0 = [P_0(1:3,3)-P_0(1:3,1)]/norm(P_I(1:3,3));
k_I0 = cross(i_I0,j_I0);
% Rotation (room → image)
R0I = [i_I0.';j_I0.';k_I0.'].

```

```

R0I = 3×3
    0.9659    0.1094   -0.2346
    0.0135    0.8837    0.4679
    0.2585   -0.4551    0.8521

```

```

% Position O_I
P1_I = P_I((1:3),1)           % in frame image

```

```

P1_I = 3×1
     0
     0
     0

```

```

P1_0 = R0I*P_0((1:3),1)       % in frame image

```

```

P1_0 = 3×1
    0.0651
    3.5791
   -1.8994

```

```

% Position vector
Oi00_I = P1_I - P1_0

```

```

Oi00_I = 3×1
   -0.0651
   -3.5791
    1.8994

```

```

% Calling ht function for Mapping between Image and Room Frames
[h_0I,h_I0] = ht(R0I,Oi00_I);
h_0I = double(h_0I)

```

```

h_0I = 4×4
    0.9659    0.1094   -0.2346   -0.0651
    0.0135    0.8837    0.4679   -3.5791
    0.2585   -0.4551    0.8521    1.8994
         0         0         0     1.0000

```

Display of Frames

```

figure
view([167.885 7.572])
plotFrame(eye(4),.7,'Room','#A2142F')
plotFrame(H_I0,.4,'Image','#0072BD')
plotFrame(H_t0,.4,'tool','#77AC30')

```



```

p = 1;
for k = 1:8
plot3(P_0(1,k),P_0(2,k),P_0(3,k),'ko', ...
'MarkerSize',3,'LineWidth',3);
end

```

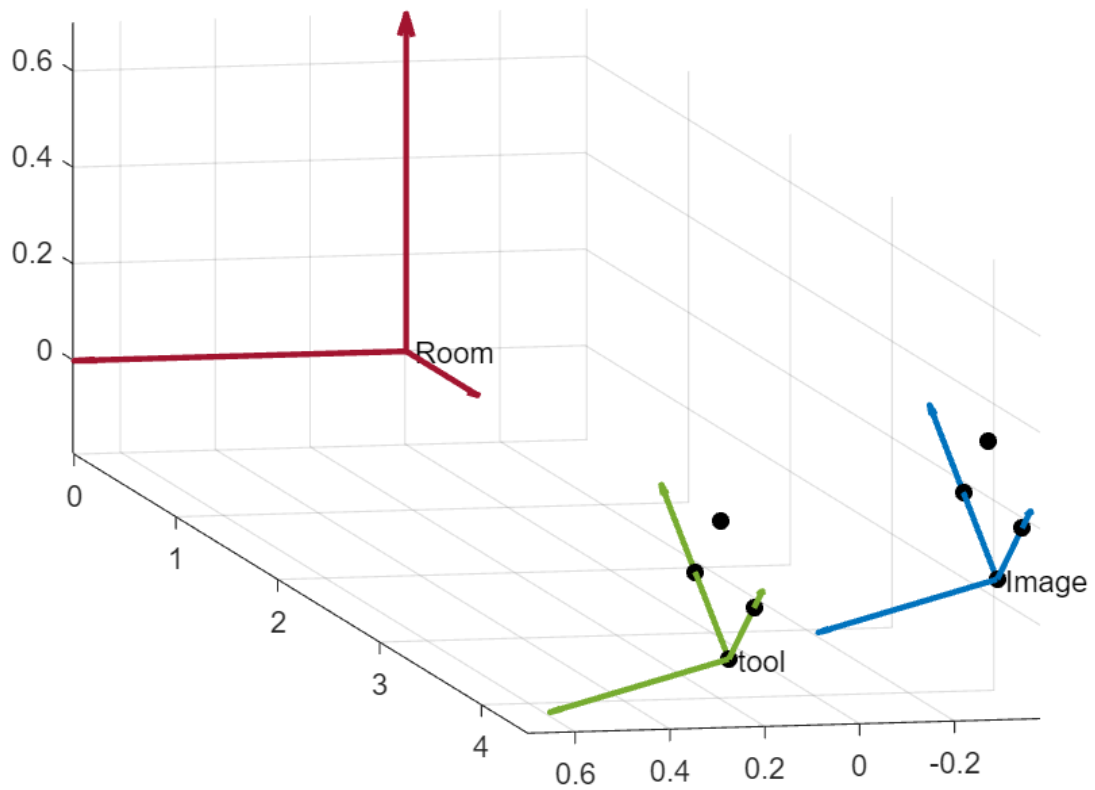


Figure 1 Coordinate frames of the room, tool, and image, with associated corner points

Appendix

Roatation functions

```
%% Rotation about x-axis
function [R_x,Rx_T] = Rx(ang)
    R_x = [1      0      0;
           0  cos(ang) -sin(ang);
           0  sin(ang)  cos(ang)];

    Rx_T = [1      0      0;
            0  cos(ang)  sin(ang);
            0 -sin(ang)  cos(ang)];

end
```

```
%% Rotation about y-axis
function [R_y,Ry_T] = Ry(ang)
    R_y = [cos(ang)  0  sin(ang);
           0      1  0;
          -sin(ang)  0  cos(ang)];

    Ry_T = [cos(ang)  0  -sin(ang);
            0      1  0;
            sin(ang)  0  cos(ang)];

end
```

Homogeneous transform function

```
%% Forming Homogeneous transform and its inverse
function [H,H_inv] = ht(Rotation,pos_vec)
    H = sym(eye(4,4));
    H(1:3,1:3) = Rotation;
    H(1:3,4) = pos_vec;

    H_inv = sym(eye(4,4));
    R_inv = Rotation.';
    H_inv(1:3,1:3) = R_inv;
    H_inv(1:3,4) = -(R_inv*pos_vec);

end
```

Function to plot Frames

```
function plotFrame(H, scale, name, clr)

if nargin < 2
    scale = 1;
end

if nargin < 3
    name = '';
end

if nargin < 4
    clr = 'b';
end
```

```

O = H(1:3,4);

x_axis = H(1:3,1);
y_axis = H(1:3,2);
z_axis = H(1:3,3);

hold on
axis equal
grid on

quiver3(O(1),O(2),O(3), ...
        scale*x_axis(1),scale*x_axis(2),scale*x_axis(3), ...
        0,'Color',clr,'LineWidth',2);

quiver3(O(1),O(2),O(3), ...
        scale*y_axis(1),scale*y_axis(2),scale*y_axis(3), ...
        0,'Color',clr,'LineWidth',2);

quiver3(O(1),O(2),O(3), ...
        scale*z_axis(1),scale*z_axis(2),scale*z_axis(3), ...
        0,'Color',clr,'LineWidth',2);

text(O(1),O(2),O(3),[' ' name]);

end

```